IT & INFORMATION SECURITY POLICIES

Secure Web Application Engineering & OWASP Defense Guide

Injection Defense, XSS Mitigation, CSRF Protection, and Security Headers

Classification: Internal **Version:** 2.9 **Owner:** Application Security (AppSec) **Standard:** OWASP Top 10 / Django Security

1. Objective and OWASP Alignment

This technical guide establishes mandatory secure coding practices for all web applications and REST APIs developed at Apex Global Technologies. Grounded in the **OWASP Top 10 Web Application Security Risks** and official **Django Security Documentation**, developers must implement these defensive controls in all application code.

2. Core Defensive Controls & Implementation Patterns

2.1 SQL Injection Prevention (OWASP A03)

Always utilize Django's Object-Relational Mapper (ORM), which automatically generates parameterized SQL queries. **Raw SQL queries using string formatting are strictly prohibited:**

```
# SECURE: Django ORM automatically parameterizes inputs
users = EnterpriseUser.objects.filter(department=department_param, is_active=True)

# FORBIDDEN: Highly vulnerable to SQL injection
# cursor.execute(f"SELECT * FROM users WHERE department = '{department_param}'")
```

2.2 Cross-Site Scripting (XSS) Mitigation (OWASP A03)

Django templates automatically escape all variables by default. Developers must never use the `|safe` template filter on untrusted user-supplied data. In addition, applications must declare strict Content Security Policy (CSP) headers:

```
# config/settings.py (via django-csp)
CSP_DEFAULT_SRC = ("'self'",)
CSP_SCRIPT_SRC = ("'self'", "https://cdn.apextech.internal")
CSP_STYLE_SRC = ("'self'", "'unsafe-inline'")
CSP_IMG_SRC = ("'self'", "data:", "https://s3.amazonaws.com")
```

2.3 Cross-Site Request Forgery (CSRF) Hardening (OWASP A01)

Ensure `django.middleware.csrf.CsrfViewMiddleware` is active in `MIDDLEWARE`. For REST APIs utilizing session authentication, verify CSRF tokens via headers:

```
CSRF_COOKIE_SECURE = True
CSRF_COOKIE_HTTPONLY = True
CSRF_COOKIE_SAMESITE = 'Strict'
```

2.4 Security HTTP Response Headers Checklist

```
# config/settings.py
SECURE_BROWSER_XSS_FILTER = True
SECURE_CONTENT_TYPE_NOSNIFF = True
X_FRAME_OPTIONS = 'DENY'
SECURE_HSTS_SECONDS = 31536000  # 1 Year HSTS
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True
SECURE_SSL_REDIRECT = True  # Enforce HTTPS redirect
```